

UGRP Meet 1

Shallow and Deep NNs - Forward Pass , Activation Functions

Krishna Murari Chivukula - ME23B233
Guides: Prof C.Balaji , Sai Krishna(PhD)

IIT Madras

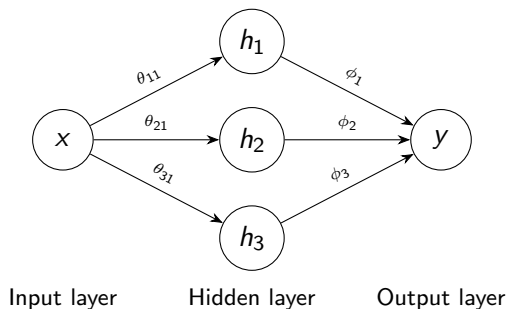
September 19, 2026

Outline

- 1 Shallow Neural Networks (SNN)
- 2 Two-input SNN
- 3 Deep Neural Networks (DNN)
- 4 Extension 1: Increasing the number of neurons
- 5 Extension 2: Different activation functions
- 6 Takeaways

Shallow Neural Networks (SNN)

- A neural network with **one hidden layer** is known as a Shallow Neural Network.
- Figure 1 shows an SNN with a single input, three hidden units, and a single output.



Forward equations

From the network diagram:

$$y = \phi_0 + \phi_1 g(h_1) + \phi_2 g(h_2) + \phi_3 g(h_3)$$

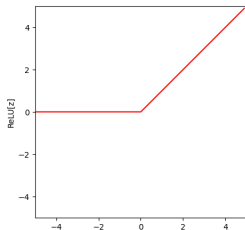
$$\Rightarrow y = \phi_0 + \phi_1 g(\theta_{10} + \theta_{11}x) + \phi_2 g(\theta_{20} + \theta_{21}x) + \phi_3 g(\theta_{30} + \theta_{31}x)$$

where $g(\cdot)$ is some non-linear activation function, and

$$h_1 = \theta_{10} + \theta_{11}x, \quad h_2 = \theta_{20} + \theta_{21}x, \quad h_3 = \theta_{30} + \theta_{31}x.$$

Let g be ReLU:

$$\text{ReLU}(x) = \begin{cases} 0, & x \leq 0 \\ x, & x > 0 \end{cases}$$



h_1, h_2, h_3 can each be positive, negative, or zero, depending on θ_{ij} and x .

Three regimes of the ReLU shallow network

Case 1 — all $h_1, h_2, h_3 > 0$

$$y = (\phi_1\theta_{11} + \phi_2\theta_{21} + \phi_3\theta_{31})x + (\phi_0 + \phi_1\theta_{10} + \phi_2\theta_{20} + \phi_3\theta_{30})$$

$y = mx + c$ — a straight line.

Case 2 — all $h_1, h_2, h_3 \leq 0$

$$y = \phi_0$$

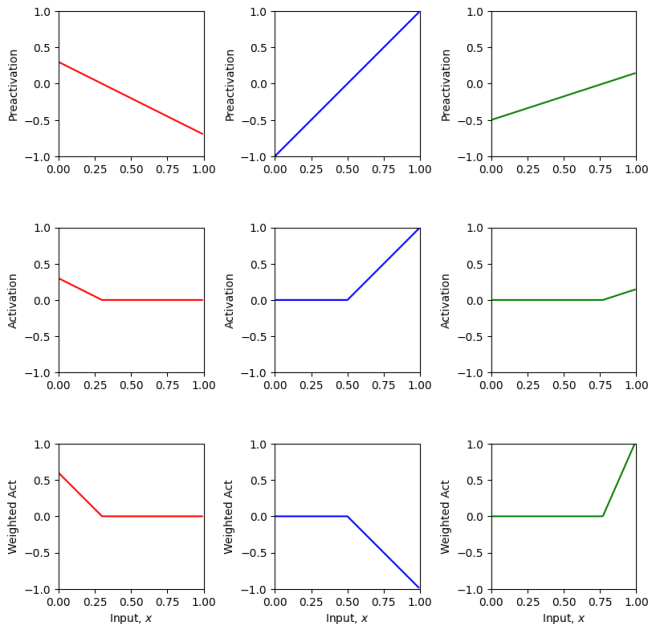
a line parallel to the x -axis.

Case 3 — only some $h_i > 0$

$y(x)$ becomes a **piecewise linear function** of x .

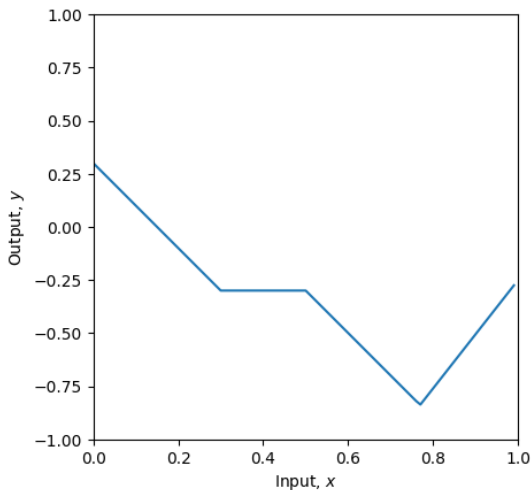
Key fact: a shallow ReLU network can exactly represent any piecewise-linear function.

Visualisation of the forward pass



Output y for the given SNN

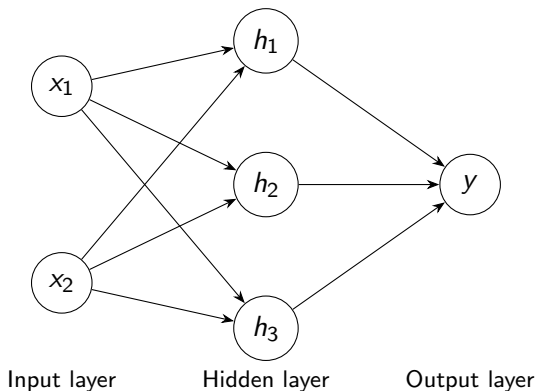
$$Y = \phi_0 + \sum_{i=1}^3 \phi_i g(\theta_{i0} + \theta_{i1}x)$$



Universal Approximation Theorem

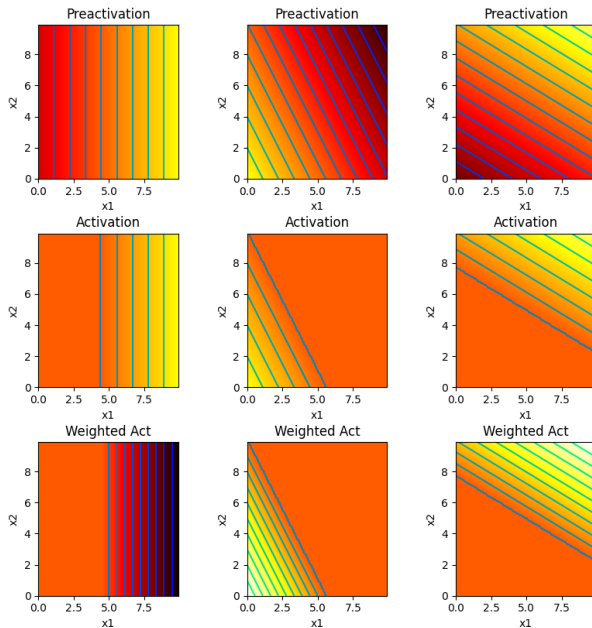
- A shallow network with ReLU gives a **continuous piecewise-linear function** with $(n + 1)$ linear regions, where n is the number of hidden units.
- Increasing n approximates the target function more accurately.
- Any continuous function can be approximated arbitrarily well by a shallow network, given enough hidden units — the **Universal Approximation Theorem**.
- However, in general it is better to use DNNs with multiple layers rather than simply increasing the number of neurons in the one layer of an SNN.

Shallow Neural Network with two inputs



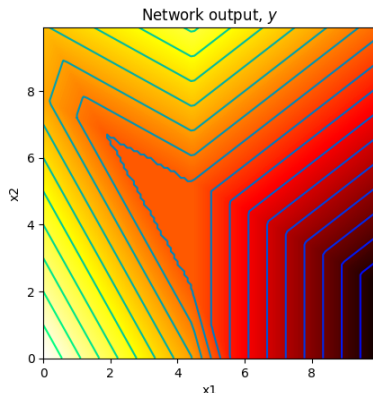
$$h_i = \theta_{i0} + \theta_{i1}x_1 + \theta_{i2}x_2, \quad Y = \phi_0 + \sum_{i=1}^3 \phi_i g(h_i)$$

Visualisation of the Forward Pass



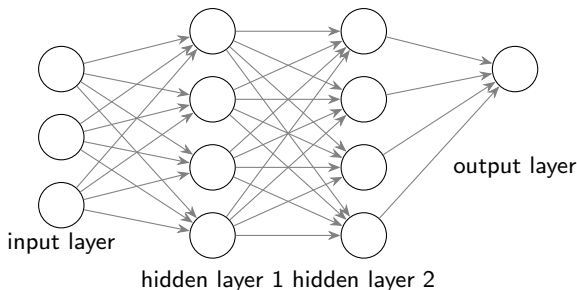
Output y for the SNN with two inputs

- A linear equation in two variables represents a **plane**; a contour plot shows the output value.
- Analogous to the single-input case, a two-input SNN represents piecewise-plane regions.
- With enough hidden units, the network can approximate any curvilinear surface.



Deep Neural Network (DNN)

- A network with **more than one hidden layer** is a Deep Neural Network.
- Highly complex curves with many deviations are impractical to fit with a single hidden layer.
- Alternative: increase the number of hidden layers.



Forward pass in a DNN

- Stacking hidden layers composes piecewise-linear (or smooth) functions of piecewise-linear functions.
- Forward-pass calculations at each layer are identical to the shallow case; only the number of layers changes.
- The resulting output function is **considerably more expressive** than a single hidden layer of the same width, for the same total parameter budget.

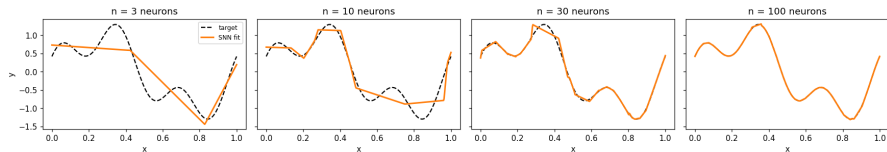
Test

Checking empirically that increasing the number of hidden neurons n lets a shallow network approximate a more complex (“wigglier”) target function.

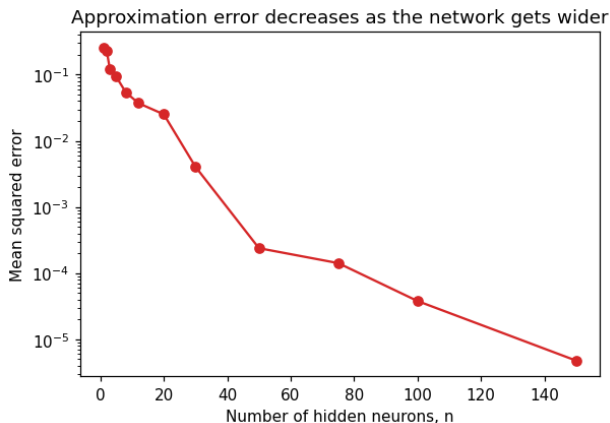
- Generalised `shallow_1_1_3` and `shallow_2_1_3` from the original notebooks to n **hidden units** using array-valued θ, ϕ .
- Hidden-layer parameters θ_{i0}, θ_{i1} sampled to spread “kinks” across the input range.
- Output weights φ solved in **closed form** via ordinary least squares (`np.linalg.lstsq`) — the same least-squares loss from the original notebook, solved directly instead of by hand.

1D: fit quality improves with width

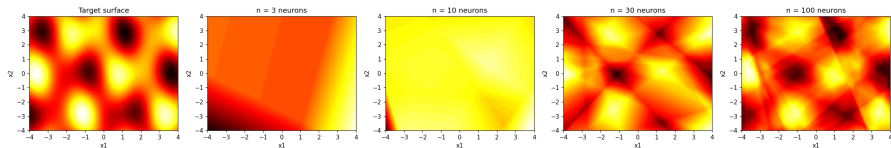
ReLU shallow network: more neurons $\rightarrow$ more piecewise-linear segments $\rightarrow$ better fit



1D: error decreases monotonically with n

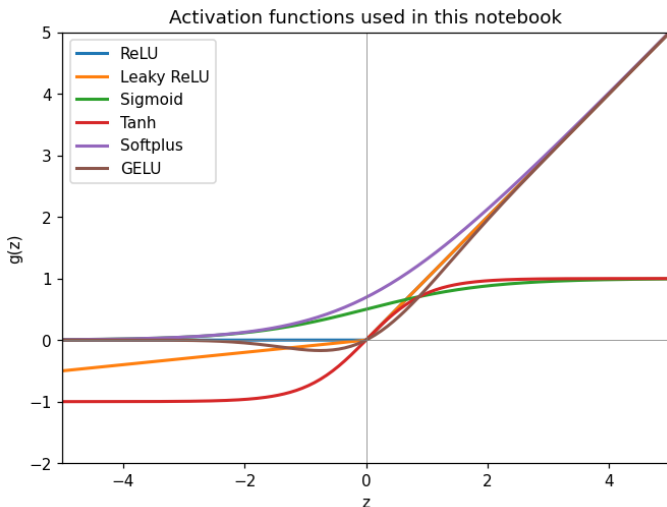


2D: fit quality improves with width



Activation functions compared

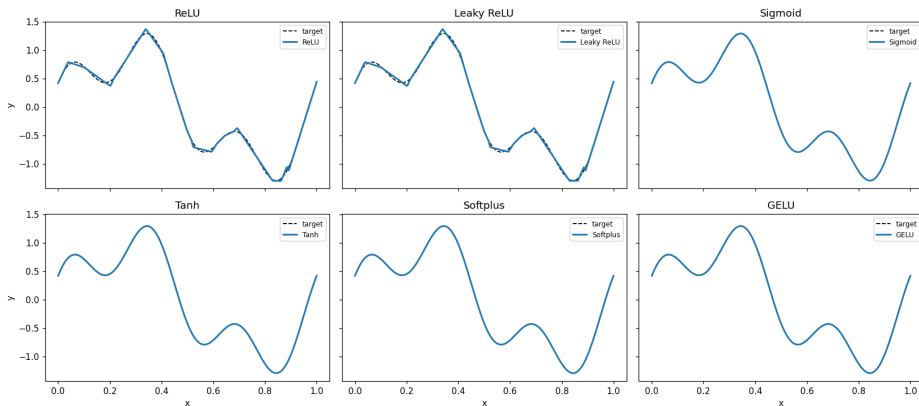
- ReLU, Leaky ReLU, Sigmoid, Tanh, Softplus, GELU.



Activation function shapes the kind of non-linearity.

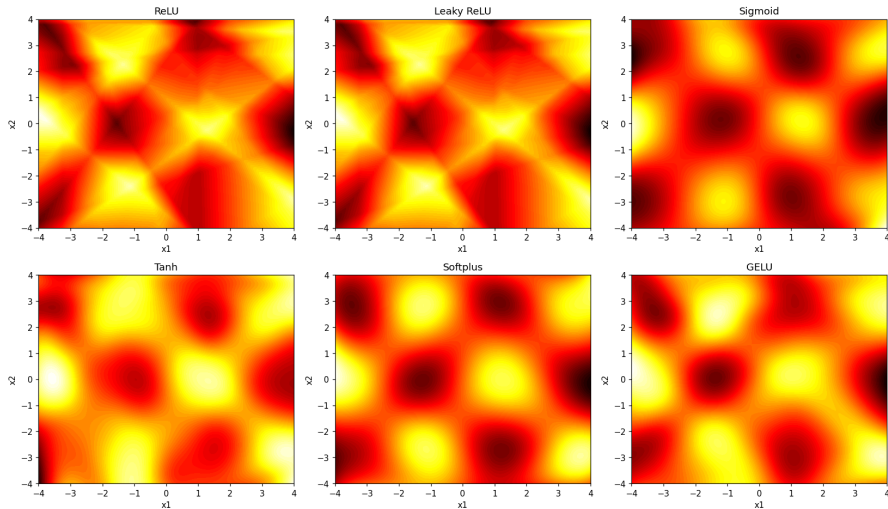
- ReLU / Leaky ReLU produce continuous piecewise-linear approximations — sharp, faceted fits
- Sigmoid / Tanh / Softplus / GELU are smooth, so the resulting fit is smooth too, with no hard kinks, at the cost of needing more units to reproduce a sharp corner exactly.

Same target, same $n=20$ neurons, six different activation functions



2D: same input and width

Same target surface, same $n=40$ neurons, six activation functions



Takeaways I

- Shallow NNs consist of 1 hidden layer with any number of neurons.
- Each layer performs a weighted linear operation followed by passing it through a non-linear activation function. This is done so that we avoid stacking linear operations to make it one big $ax + b$ type function.
- ReLU is most commonly used as an activation function , and a shallow NN with ReLU can approximate any piecewise linear function.
- Using wide SNNs(more neurons) or DNNs we can extend this linear approximation to any general function , which is the(as long as we are using an appropriate non linear activation function) , which is the universal approximation theorem.
- **Width(number of neurons in a layer) is useful** For a fixed activation, more hidden neurons $\Rightarrow$ more pieces to work with $\Rightarrow$ lower approximation error, which is the practical result of the UAT.

- **Activation shapes the kind of non-linearity:**
 - ReLU / Leaky ReLU $\rightarrow$ sharp, faceted, piecewise-linear fits.
 - Sigmoid / Tanh / Softplus / GELU $\rightarrow$ smooth fits, no hard kinks, but need more units to reproduce a sharp corner.
- Going **deep** (stacking hidden layers) composes these non-linear pieces further, giving much greater expressive power per parameter than simply widening a single hidden layer.